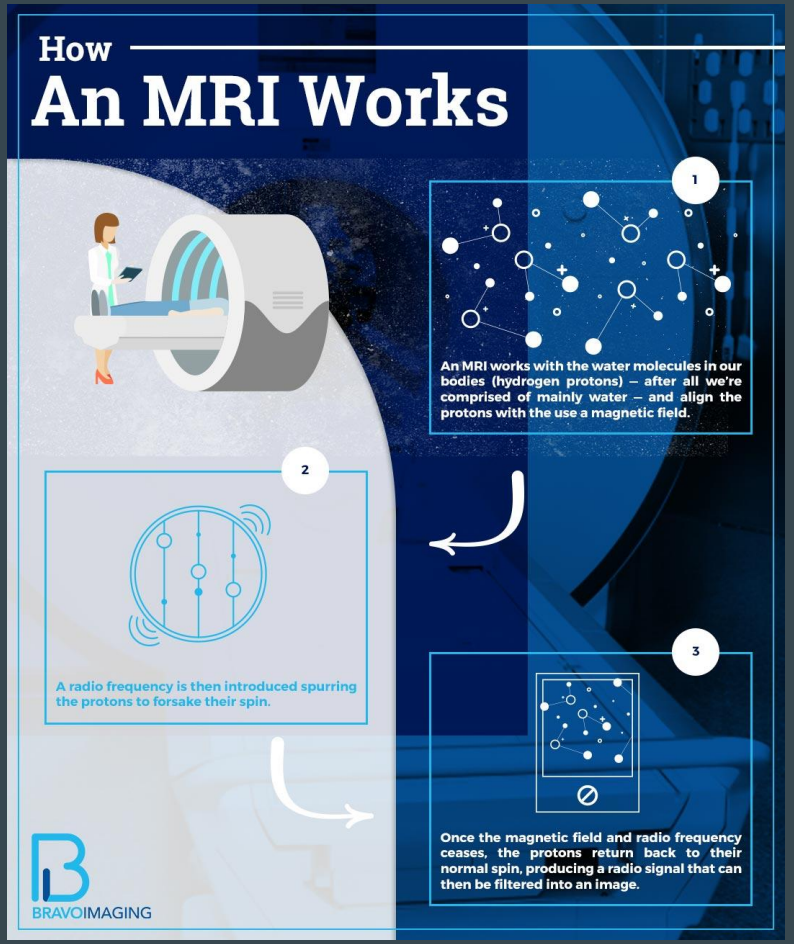
# MRI Reconstruction

...

# Background

MRI machines don't actually take pictures: they use magnets and the physical properties of water in our bodies to capture radio-frequency data.

Each radio signal is converted to a pixel in an image using a Fourier transform. By convention, we say that the MRI stores radio-frequency data in *k-space*, before being converted to an image.




Source: Bravo Imaging

# The Problem

There are a lot of pixels in a typical MRI image, which means the machine has to send out a lot of radio-frequencies, which **takes a long time** . This is **expensive** for hospitals operating MRI machines and **uncomfortable** for patients, who have to stay still while images are taken.

Staying still for a long time is hard for

- Children
- Dementia Patients
- Patients with Huntington's, addictions, etc.

Shorter scan times = more accessible treatment

# Potential Solution

What if, instead of measuring every value in k-space, we only directly measured a subset of them?

- Quicker scans, but more blurry images

Can we can **reconstruct** a partially formed MRI image using optimization?

- Quick scans with more detailed images

# Mathematics

A = MF

- M = Diagonal Matrix (entries are 1 if frequency is being measured, 0 if not)
- F = Discrete Fourier Transform

Given observed undersampled k-space data *y_obs,* we are looking for $\alpha$ that solves

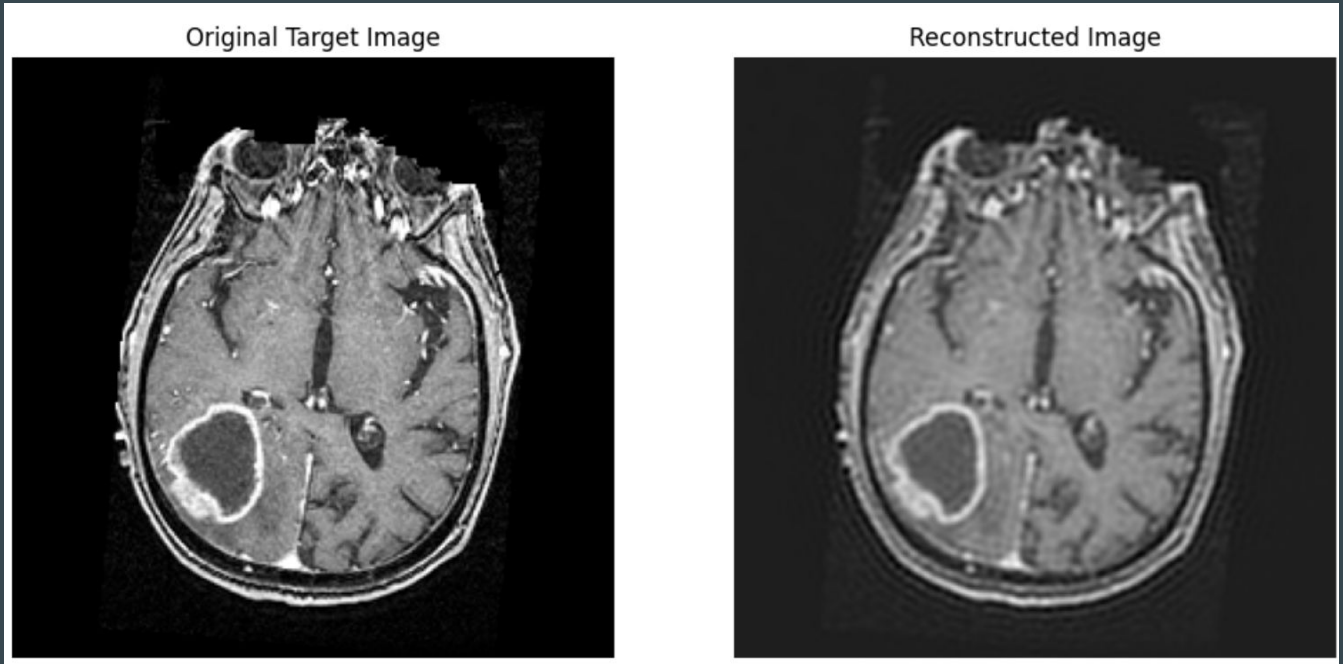
$$\min_{\alpha \in \mathbb{R}^n} ||A(K\alpha) - y_{obs}||_2^2$$

There are many possible solutions, so we restrict the reconstruction to a class of kernel basis functions for uniqueness. We use **Gaussian kernels** to impose smoothness.

# Implementation

- **Data**: UPenn-GBM Cancer Imaging Archive. Selected target image.
  - Create kernel basis & lots of data preparation
- **Simulate** the under-sampling of frequencies: ‘masking’ the data by adding a filter to only show low-frequency waves.
- **Solve**: Using an Adam (Adaptive Moment Estimation) optimizer.
- **Validate** : (1) compare reconstructed image to original MRI target image and (2) measure loss over iterations.

# Results




Pro:

- Working prototype w/real data

Con:

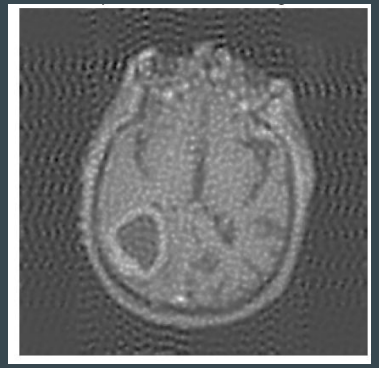
- Blurry image
- No frame of reference for accuracy

# Comparing Kernels vs. Baseline Reconstruction

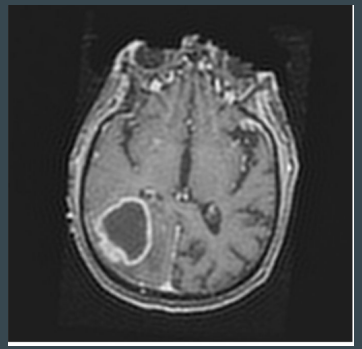
| *Metric* | *LaPlacian Kernels* | *Gaussian Kernels* | *Zero-Filled FFT* |
|---|---|---|---|
| *MSE* | 0.0186 | 0.00529 | 0.010049 |
| *PSNR* | 17.30 dB | 22.78 dB | 19.98 dB |

Original Image

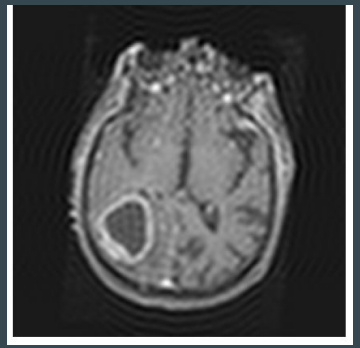
Laplace Reconstruction



Gaussian Reconstruction



Zero-filled FFT Reconstruction

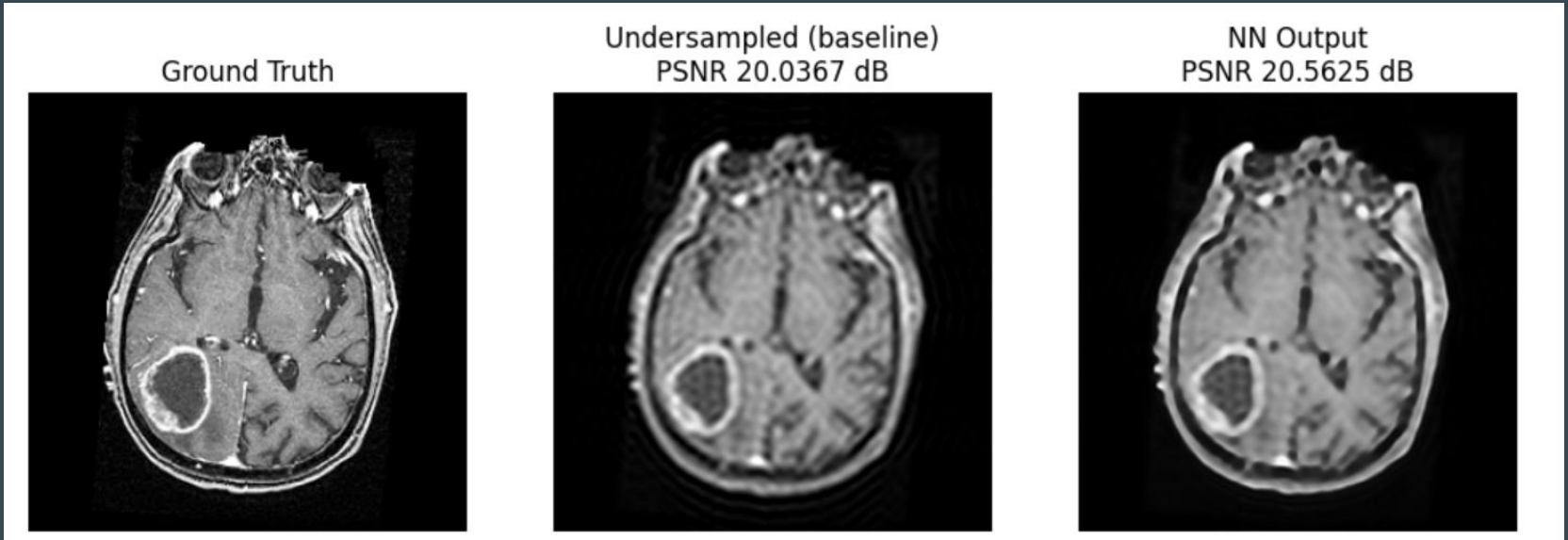


Gaussian kernel reconstruction provides highest clarity.

# Another Baseline: The Traditional Approach

- Modern MRI reconstruction often uses learned priors (CNNs/diffusion models)
- Trained a small CNN:
  - 3 layers, 64 channels, 3x3 kernels (standard, small)
  - Input: undersampled MRI slice
  - Predicts a correction: $x* = x + f(x)$where
    - $x^*$ is final reconstructed image
    - $f(x)$ is learned correction
    - $x$ is the undersampled input
  - Used L1 loss (common in image restoration tasks): $||x + f(x) - x_{true}||$

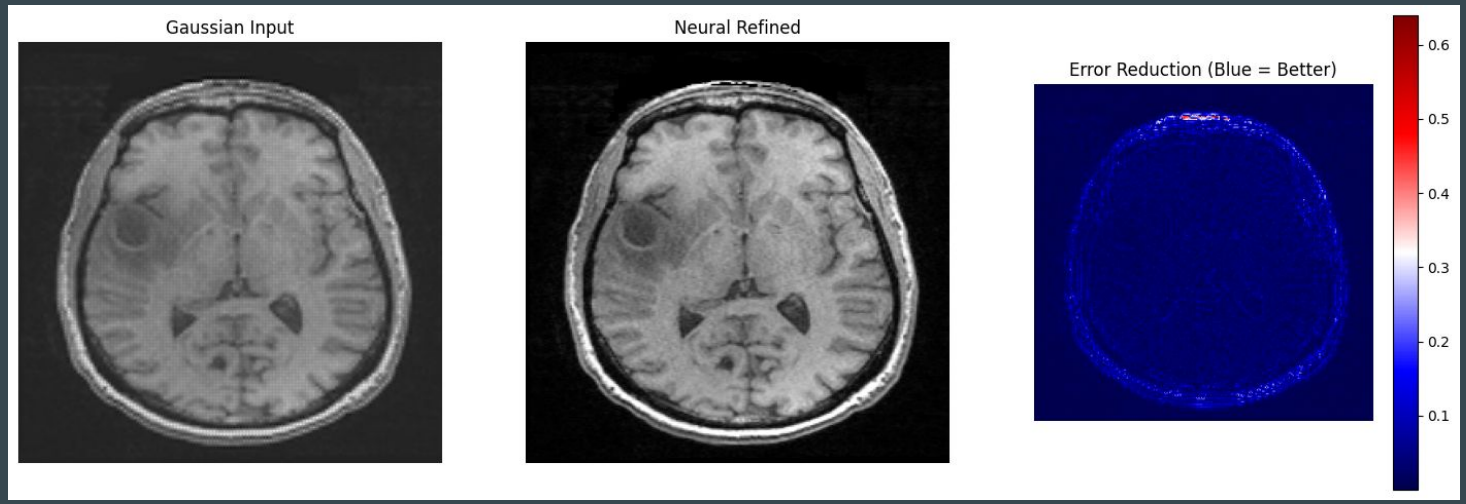
# Neural Network Baseline Results




Gaussian kernel still achieves highest PSNR, implying the kernel method is competitive to traditional, learned methods.

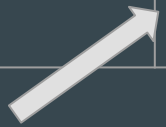
# Warm-Start Neural Network

Again, used kernel reconstruction as input to a neural network but only let the neural network fill in the directions the kernel did not already smooth. We also changed the neural network architecture to U-Net, rather than ReLU which is more common in medical applications.



# Warm-Start Neural Network Results

| | Gaussian Reconstruction | Warm-Start NN Reconstruction |
|---|---|---|
| PSNR | 19.581 dB | 137.75 dB |
| MSE | 0.011 | 0 |
| SSIM | 0.605 | 1.0 |



Achieved a perfect reconstruction of the desired image!

# Super Resolution Convolutional Neural Network (SRCNN)

SRCNN is a three-layer CNN for single-image super resolution. Instead of applying kernels, we used SRCNN to reconstruct the entire slice. It made the following improvements:

| | Kernel Optimizer (Gaussian) | SRCNN | Improvement |
|---|---|---|---|
| PSNR | 28.78 dB | 37.74 dB | 8.96 dB |
| SSIM | 0.8786 | 0.9534 | 0.0748 |

# Next steps

- **Train / test across multiple slices:** so far, we have mostly been testing on a single slice to show proof of concept. Now we need to test across multiple slices
- **Diffusion models:** see if replacing the warm-start neural network with diffusion model improves reconstruction since diffusion models used in image generation
- **Multi-image super resolution:** add context by considering the slices before/after the slice that is being reconstructed
- **Images & Videos** : try our reconstruction approaches on various images and videos

# Multi-Slice Training: Setup

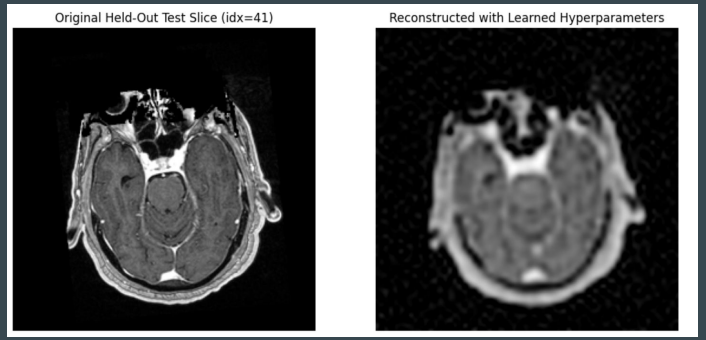
- **Adding a TV penalty:** Our optimization equation largely remained the same, with the addition of a TV regularizer:
- **Parameter Tuning**
  - Training for kernel width, number of kernels, and TV penalty.
  - Using both Gaussian and LaPlacian kernels to align with literature.
- **Training Steps** :
  - Choose candidate hyperparameters,
  - Reconstruct a set of training slices with those hyperparameters
  - Compute reconstruction metrics for each slice.
  - Average metrics across the slices and selected the hyperparameters that performed the best on the test slices.

# Multi-Slice Training

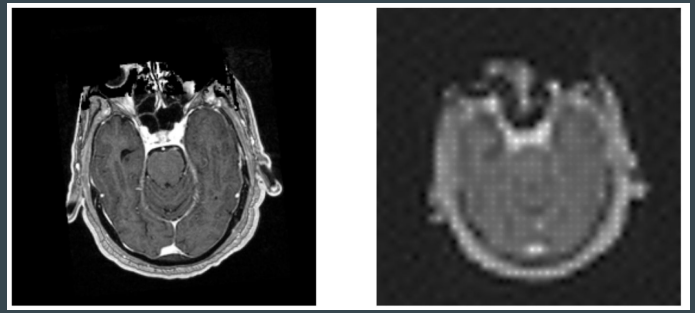
| Kernel Type | Sigma | Number of Kernels | TV Weight | Test MSE | Test PSNR |
|---|---|---|---|---|---|
| LaPlacian | 4.1017 | 1600 | 1.3145e-03 | 0.009615 | 20.173 dB |
| Gaussian | 2.5 | 3844 | 0.0 | 0.007886 | 21.032 dB |

Like in the single-slice model, the Gaussian kernels performed better as measured by MSE and PSNR.

LaPlacian

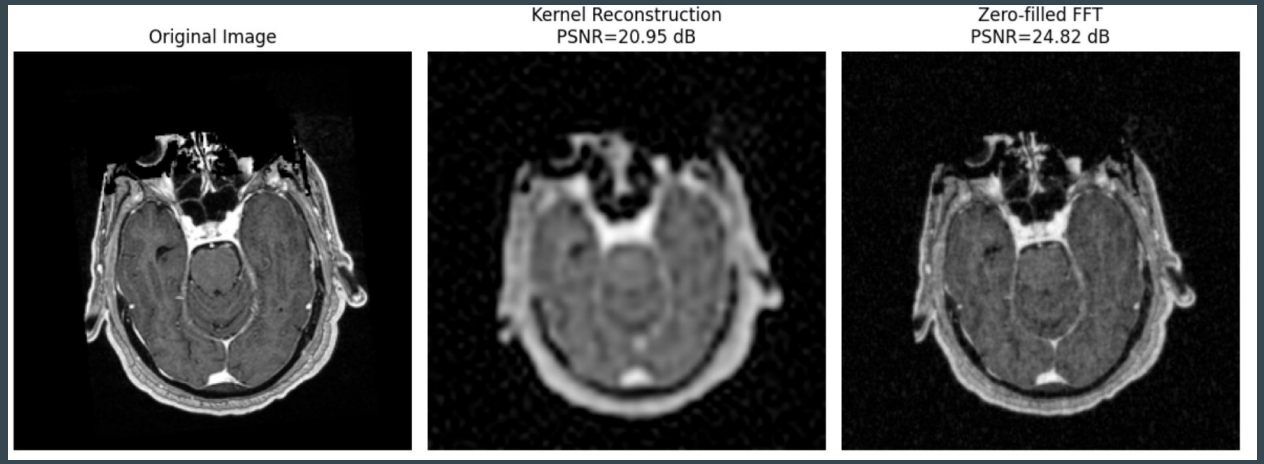



Gaussian



# Multi-Slice Training: Zero-Filled FFT

- Lower PSNR for multi-slice zero-filled FFT compared to the single-slice model.
- Simpler method with lower computational time makes zero-filled a good candidate for more advanced models.



# Residual Convolutional Neural Networks (CNN) - Setup

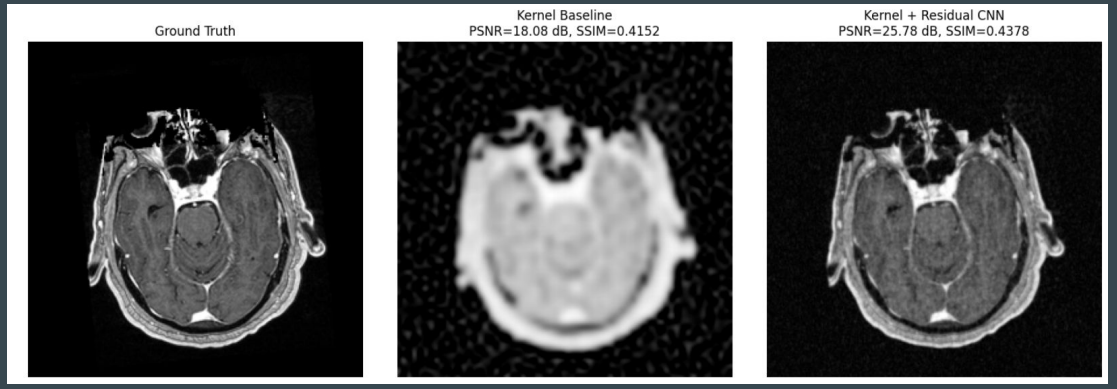
Basic setup: $x_{output} = x_{input} + f(x_{input})$

- E.g. CNN output = Warm-start (from zero-filled FFT) + learned residual correction
- Restriction: Model cannot change measured k-space values
- Tested with zero-filled FFT, Gaussian kernels, and LaPlacian kernels.

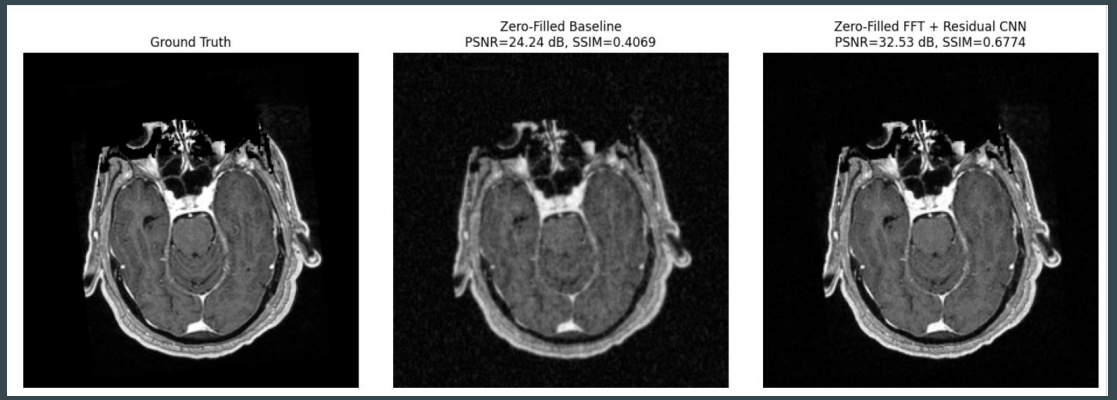
| | Test MSE | Test PSNR | Test SSIM |
|---|---|---|---|
| Kernel Reconstruction & Residual CNN | 0.002479 | 26.06 dB | 0.4153 |
| Zero-filled FFT Reconstruction & Residual CNN | 0.000478 | 33.20 dB | 0.6758 |

# Residual CNN - Results

Kernel Reconstructions




Zero-Filled FFT Reconstructions




- Residual CNN consistently shows better performance than kernel reconstruction alone
- Zero-filled FFT outperforms kernel reconstruction on PSNR, MSE, SSIM, and compute time.

# Diffusion Models

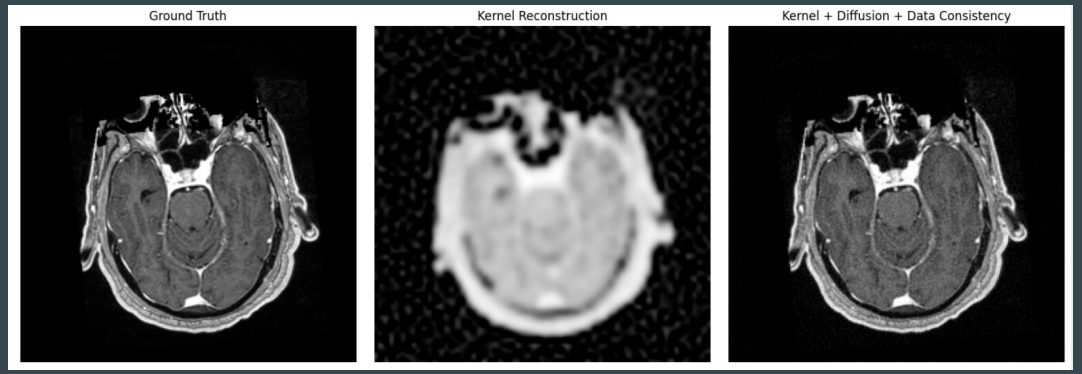
Diffusion models are commonly used in image reconstruction by iteratively adding noise to training images and producing image output from gradually removing noise layers from a pure-noise image to recover the original image.

- Step 1: Train diffusion model on the initial reconstruction (kernel or zero-filled FFT)
- Step 2: Model iteratively corrects the areas of the image that were not measured k-space data (i.e., “real” parts of the image).

| | Test MSE | Test PSNR | Test SSIM |
|---|---|---|---|
| Kernel Reconstruction & Diffusion Model Correction | 0.000690 | 31.61 dB | 0.7796 |
| Zero-filled FFT Reconstruction & Diffusion Model Correction | 0.000772 | 31.12 dB | 0.7620 |

# Diffusion Model: Results

Kernel Diffusion Model




Zero-Filled FFT Diffusion Model


MSE was lower and PSNR and SSIM were higher for the kernel reconstruction compared to the zero-filled FFT, showing that the **combination of the kernel reconstruction and diffusion model actually does better than the baseline approach.**

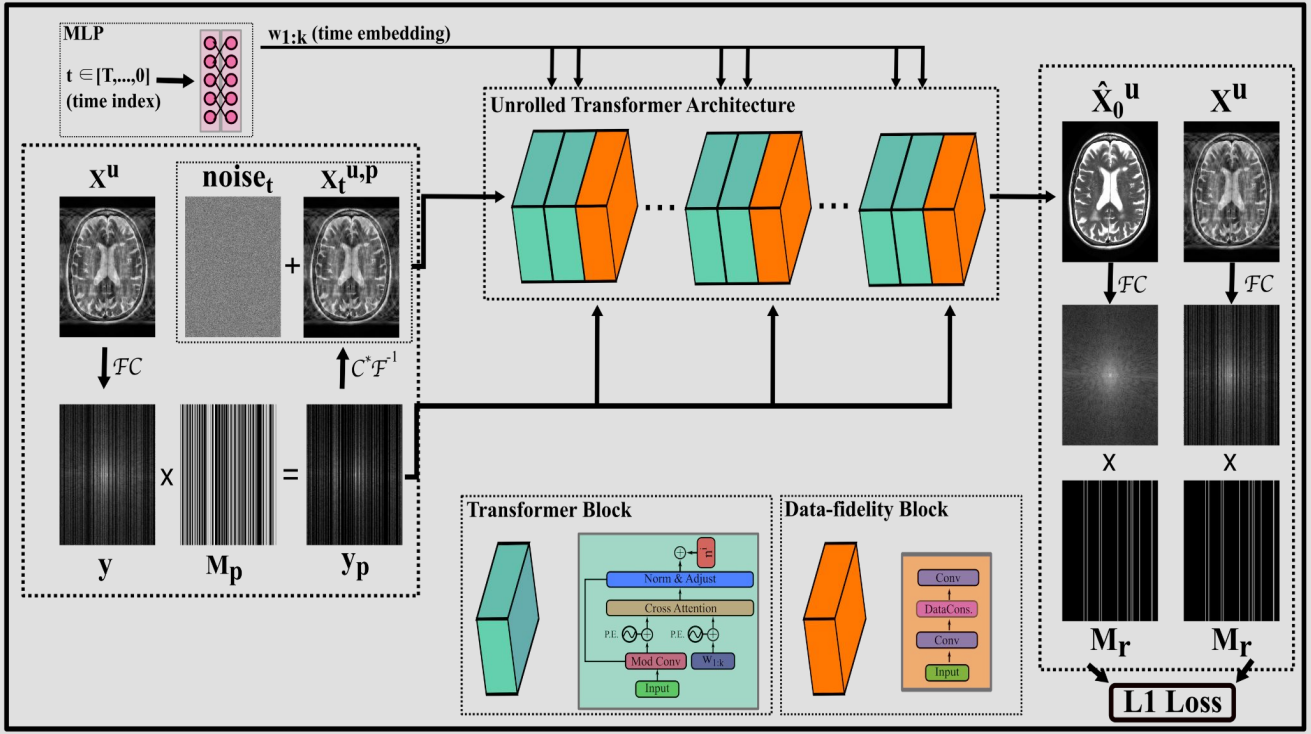
# Self-Supervised (Self-Tuning) Diffusion Model

How does the model perform with less available information?

Simulate with a **split k-space masking strategy** .

**M** = the undersampled mask, **Mr (train)** = random sample of M (= 8% here)

**Mp = M / Mr (test)**




Based on paper by Korkmaz et. al (2024)

# Self-Supervised Diffusion Model Results


Model is not making changes from zero-filled FFT because it is solving a difficult optimization problem.

Can we change the setup to force the model to make changes?

| Loss | MSE | SSIM | PSNR |
|---|---|---|---|
| 0.074 | 0.002 | 0.427 | 26.12 dB |

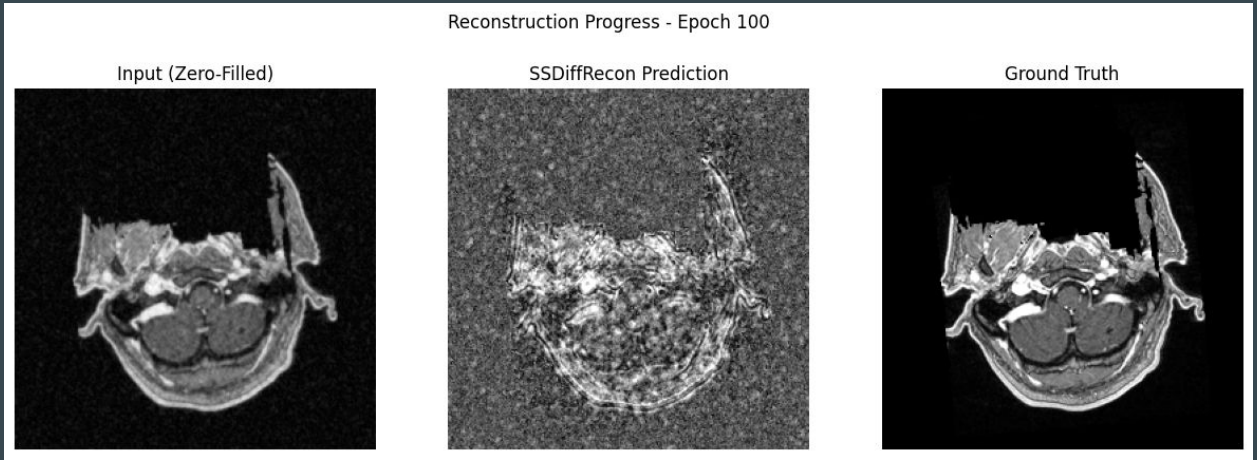
# Self Supervised Diffusion Model - attempt 2 (failed)

**Step 1: Frequency Weighted Loss** : We implemented focal frequency loss because the model had a lot of trouble initially **filling in details** - rather, it duplicated the input with minimal changes -> now the whole screen is black

**Step 2: Sigmoidal activation function:** Constrains the output to the [0,1] range, preventing extremely bright points in k-space from dominating the image and making model improvements clearer -> better, grainy output but rough shape

**Step 3: Cosine Annealing Scheduler with warmup** : We start with a **linear scheduler** before switching to cosine annealing so that the model can learn the general shape of the image before dropping the learning rate -> still not great!

# Self-Supervised Diffusion Model - version 2 Results

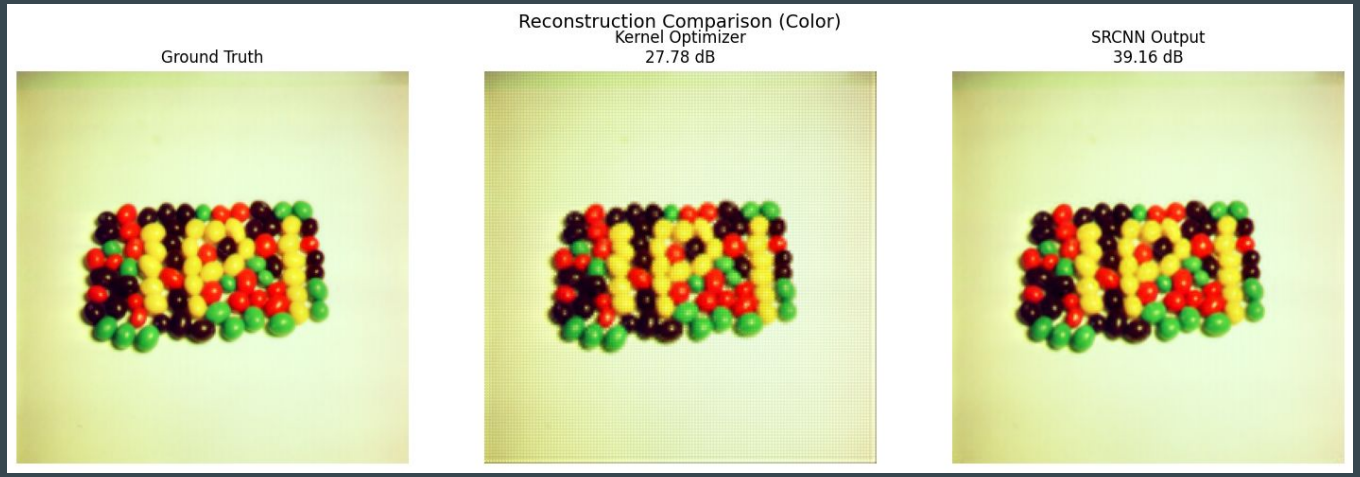



Sigmoidal function really worsened the output!

Did not achieve same results as paper due to smaller training dataset and smaller epoch length.

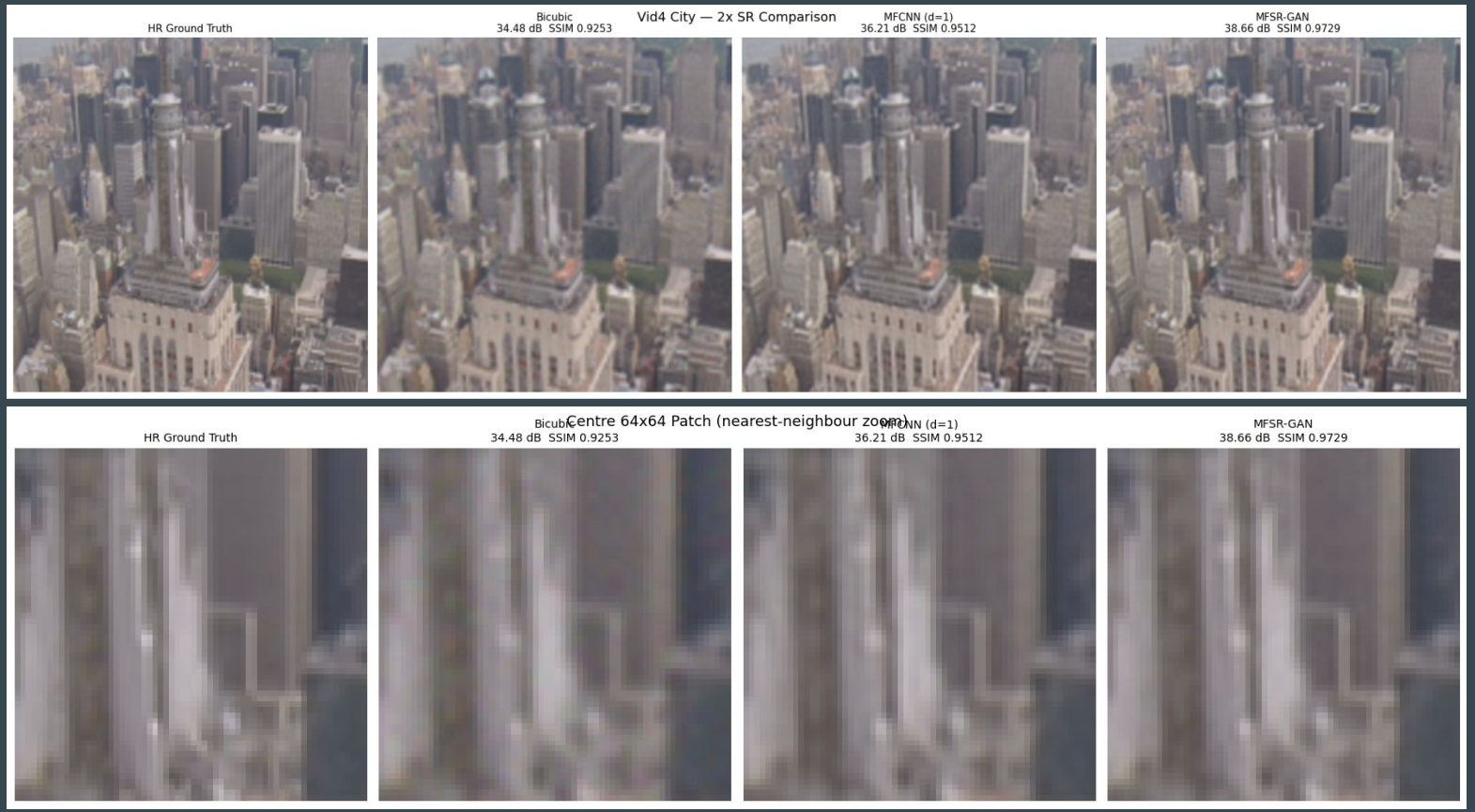
| Loss frequency-weighted | MSE | SSIM | PSNR |
|---|---|---|---|
| 0.167 | 0.110 | -0.05 | 9.59 dB |

# Colored SRCNN




Updated the SRCNN to work with color and currently testing on deeper levels of SRCNN and training on multiple images

# Multiframe CNN Methods



# Multiframe CNN Methods(Continued)

We have implemented two types of Multiframe CNN methods which take into account the context of neighboring frames to help strengthen the reconstruction of each frame

**MFCNN (Greaves & Winter, 2016)**

A lightweight multi-frame CNN approach. All 5 LR burst frames are bicubic-upsampled to HR size, then concatenated along the channel axis (giving 15 channels for a d=1 window). A simple 5-layer conv net maps this to the SR output.

**MFSR-GAN (Khan et al., CVPRW 2025)**

A more sophisticated GAN-based approach with:

- Reference Difference Computation, deformable convolutions for motion-aware alignment, channel attention for multi-frame features, RRDB (Residual-in-Residual Dense Block), trained with a relativistic GAN loss + L1 loss

This initial test was done on a standard Vid4 “city” benchmark where each frame was downsampled and had noise added to simulate a blurry image

The next steps are to apply these methods to MRI imaging to see if they improve reconstruction as well as training the model on more data

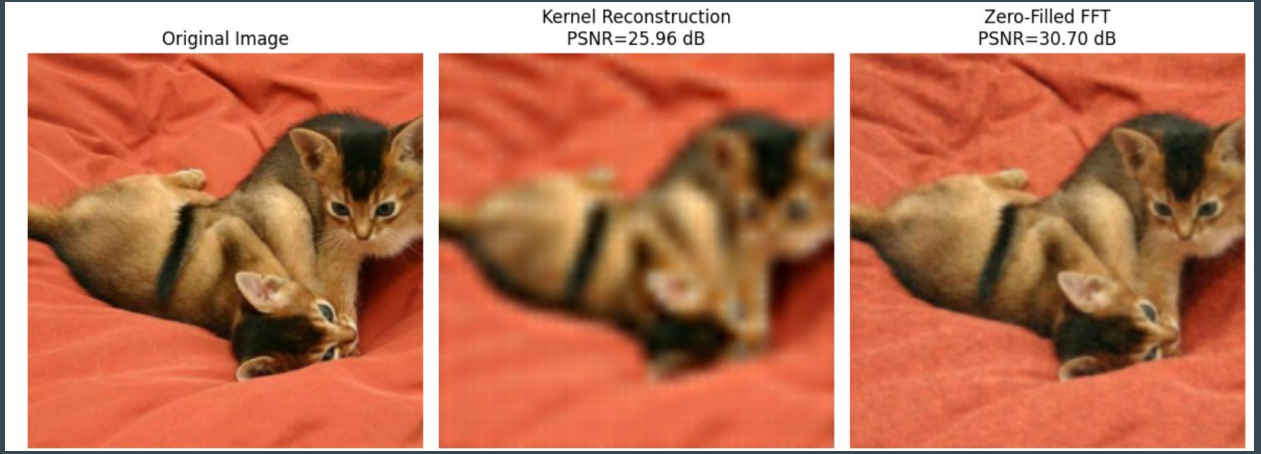
# General Images

# Hyperparameter Tuning

| | LaPlacian | Gaussian |
|---|---|---|
| Sigma | 7.0 | 12.0 |
| Number of Kernels | 1600 | 3600 |
| TV Weight | 1.0e-04 | 1.0e-02 |
| MSE | 0.004535 | 0.008821 |
| PSNR | 24.196 dB | 21.242 dB |

LaPlacian achieves lower MSE and higher PSNR, differing from MRI images.

# Initial Reconstruction



# Kernel & Residual CNN Reconstruction


# Zero-filled FFT & Residual CNN Reconstruction

# Residual CNN Results

| | Kernel Reconstruction & CNN | Zero-filled FFT Reconstruction & CNN |
|---|---|---|
| MSE | 0.000833 | 0.000382 |
| PSNR | 30.79 dB | 34.18 dB |
| SSIM | 0.8540 | 0.9104 |

Zero-filled FFT still achieved better reconstruction, similar to MRI reconstruction results

# Kernel & Diffusion Model Reconstruction


# Zero-filled FFT & Diffusion Model Reconstruction

# Diffusion Model Results

| | Kernel Reconstruction & Diffusion | Zero-filled FFT & Diffusion |
|---|---|---|
| MSE | 0.004658 | 0.004504 |
| PSNR | 23.32 dB | 23.46 dB |
| SSIM | 0.4506 | 0.4564 |

Zero-filled FFT still achieved better results, differing from the MRI reconstructions

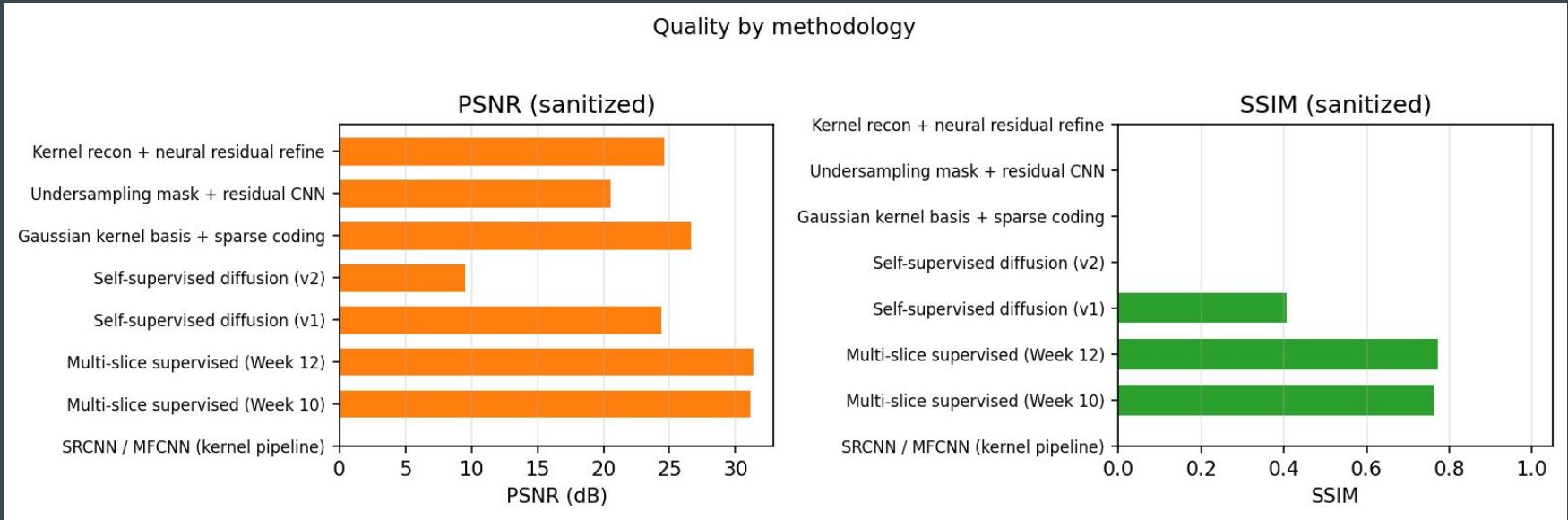
Both methods achieved worse results than the residual CNN

# Next Steps

- Self-tuning diffusion models: to further reduce noise floor
- Multi-frame super resolution models: allows model to use context from neighboring images in the reconstruction
- Extension to non-medical imaging: Providing better baseline
- Improving zero-filled FFT instead of the kernel reconstruction
- Training SRCNN and MFCNN on more data

# Autoresearch

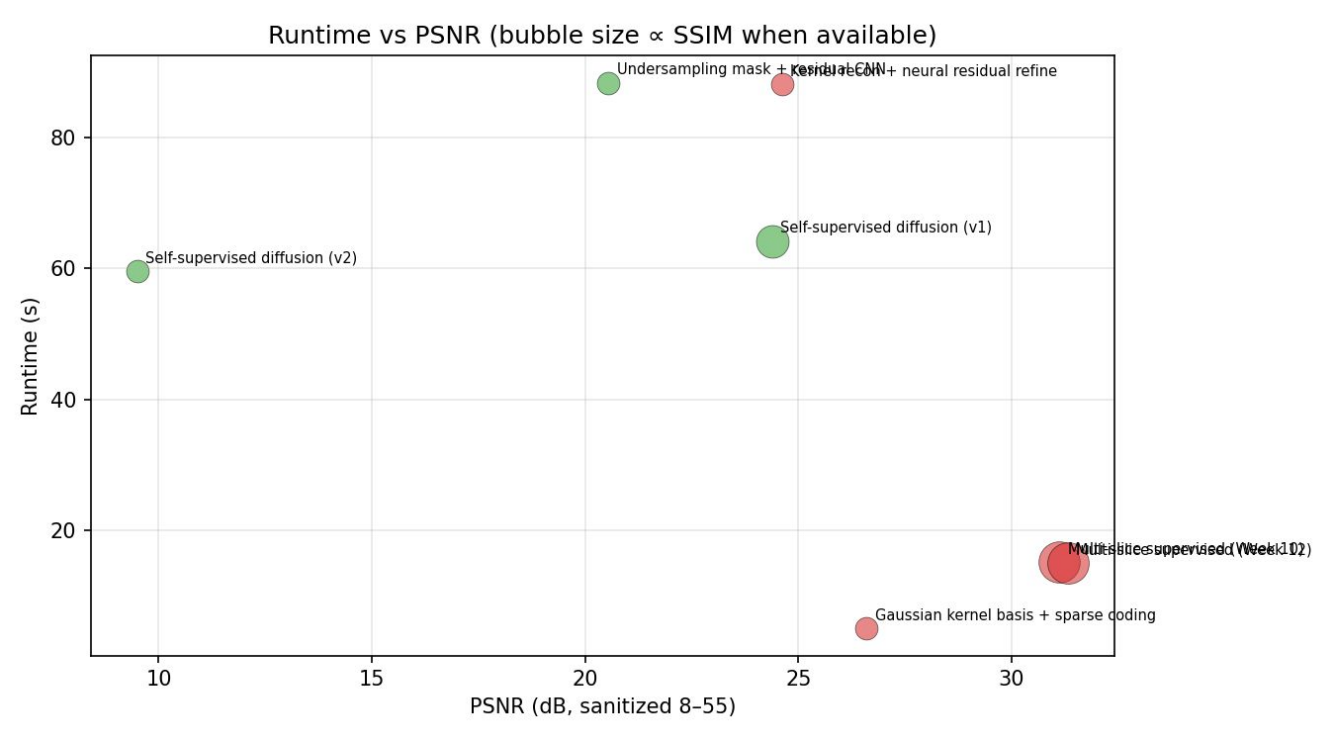
# Model Comparison: PSNR and SSIM



# What PSNR & SSIM Show

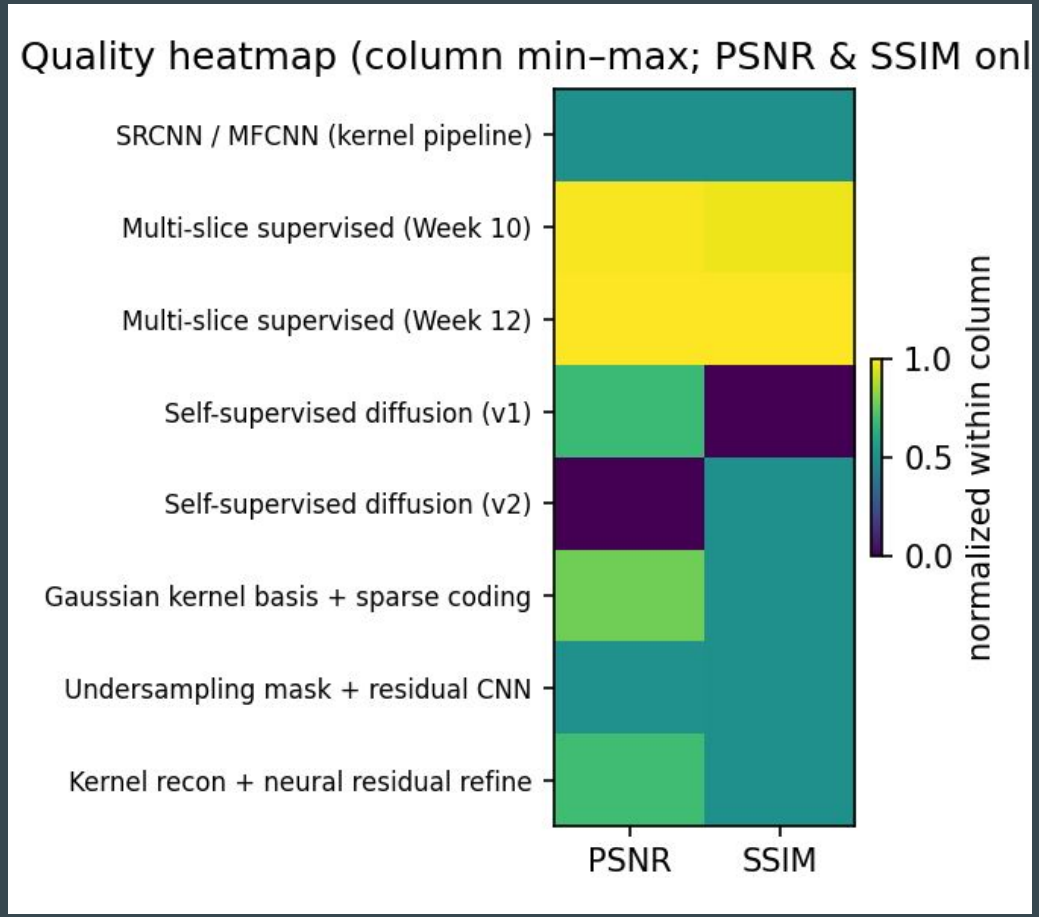
- Multi-slice supervised models lead on both metrics
- SSIM data is missing for several methods, making direct comparison incomplete
- PSNR and SSIM capture different qualities — one number isn't enough

# Model Comparison: Runtime vs. Success Metrics



# Model Efficiency

- Multi-slice supervised: high PSNR and low runtime (~15s)
- Diffusion models: long runtime (~60-90s) for middling PSNR - costly tradeoff
- Self-supervised diffusion v2: worst of both worlds - slow and lower quality
- For real deployment, multi-slice supervised is the most practical choice
- Diffusion models need significant optimization before clinical use




No model dominates on both PSNR and SSIM simultaneously

Self-supervised diffusion v1 flips - relatively strong PSNR, weak SSIM

Multi-slice supervised is the most consistent across both metrics

Confirms why using MSE + PSNR + SSIM together matters — each catches something different

Future work: find or design a model that scores well on all metrics, not just one